

分类号_____

编号_____

UDC_____

密级_____



南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

本科生毕业设计（论文）

题 目：基于预测模型的多 LoRA 适配器推理优化研究

姓 名：施米乐

学 号：12212921

院 系：计算机科学与工程系

专 业：计算机科学与技术

指导教师：李卓钊

2026 年 5 月 7 日

诚信承诺

1. 本人郑重承诺所呈交的毕业设计（论文），是在导师的指导下，独立进行研究工作所取得的成果，所有数据、图片资料均真实可靠。

2. 除文中已经注明引用的内容外，本论文不包含任何其他人或集体已经发表或撰写过的作品或成果。对本论文的研究作出重要贡献的个人和集体，均已在文中以明确的方式标明。

3. 本人承诺在毕业论文（设计）选题和研究内容过程中没有抄袭他人研究成果和伪造相关数据等行为。

4. 在毕业论文（设计）中对侵犯任何方面知识产权的行为，由本人承担相应的法律责任。

作者签名：

_____年____月____日

基于预测模型的多 LoRA 适配器推理优化研究

施米乐

(计算机科学与技术专业, 指导老师: 李卓钊)

【摘要】：随着大语言模型的广泛应用，基于 PEFT (Parameter-Efficient Fine-Tuning) 的技术，如 LoRA (Low-Rank Adaptation)，被广泛用于支持多任务与个性化推理。在实际部署中，一个推理系统往往需要同时服务大量 LoRA 适配器，而 GPU 显存限制使得系统无法同时加载全部适配器，从而需要在运行过程中频繁进行适配器的加载与替换。该过程会引入显著的性能开销，成为多 LoRA 推理系统的关键瓶颈。

现有方法通常采用被动的适配器管理策略，例如基于 LRU 或 LFU 的缓存机制。这类方法仅依赖历史访问信息进行决策，缺乏对未来请求的建模能力，在动态或突发负载场景下容易导致频繁的缓存失效与适配器切换。

为了解决上述问题，本文提出一种基于预测的适配器调度方法。该方法利用指数滑动平均 (EMA) 对适配器访问趋势进行建模，并结合请求队列信息与适配器活跃状态构建统一的效用函数，同时引入显存感知机制对不同适配器进行归一化处理。在显存约束下，系统通过求解近似的效用最大化问题动态维护适配器驻留集合，从而优先保留高价值适配器。

【关键词】：LoRA 推理，适配器调度，预测驱动缓存，指数滑动平均，资源管理

[ABSTRACT]: With the rapid adoption of large language models (LLMs), parameter-efficient fine-tuning (PEFT) methods such as LoRA (Low-Rank Adaptation) have been widely used to support multi-task and personalized inference. In practical deployment scenarios, a serving system often needs to manage a large number of LoRA adapters simultaneously. However, due to limited GPU memory, it is impossible to keep all adapters resident on GPU at the same time, resulting in frequent adapter loading and replacement during runtime. This process introduces significant overhead and becomes a major performance bottleneck in multi-LoRA serving systems.

Existing approaches typically adopt passive adapter management strategies, such as LRU- or LFU-based caching mechanisms. These methods rely only on historical access information and lack the ability to model future requests, making them prone to frequent cache misses and adapter switching under dynamic or bursty workloads.

To address these challenges, this paper proposes a prediction-based adapter scheduling method. The proposed approach utilizes Exponential Moving Average (EMA) to model adapter access trends, and further incorporates request queue information and adapter activity states into a unified utility function. In addition, a memory-aware normalization mechanism is introduced to handle heterogeneous adapter sizes. Under GPU memory constraints, the system dynamically maintains the resident adapter set by approximately solving a utility maximization problem, thereby prioritizing high-value adapters and reducing unnecessary adapter reloads.

[Keywords]: LoRA inference, adapter scheduling, prediction-driven caching, exponential moving average, resource management

目录

1. 引言.....	5
2. 系统模型与问题定义	5
2.1 系统模型	5
2.2 问题定义	6
2.3 基线方法	7
2.3.1 FIFO (First-In, First-Out)	7
2.3.2 LRU (Least Recently Used)	7
2.3.3 LFU (Least Frequently Used)	7
2.3.4 方法对比分析	8
3. 方法设计	8
3.1 方法概述	8
3.2 基于 EMA 的适配器使用预测	8
3.3 适配器效用建模	9
3.4 显存感知的效用归一化.....	9
3.5 动态适配器选择策略	10
3.6 在线决策过程.....	10
4. 实验设计	10
4.1 实验目标	10
4.2 实验设置	11
4.3 工作负载	11
4.3.1 长尾分布	11

4.3.2 动态热点迁移	12
4.3.3 异构适配器	12
4.4 实现细节	13
4.5 数据分析	14
4.5.1 总体表现	14
4.5.2 消融实验	15
5. 相关研究	16

1. 引言

随着大语言模型的广泛应用，基于 LoRA 的技术，被广泛用于支持多任务与个性化推理。然而，在实际部署中，一个推理系统往往需要同时服务大量 LoRA 适配器，每个请求依赖于不同的适配器，这对系统的资源管理能力提出了新的挑战。

在多 LoRA 推理场景中，通常仅有一个基础模型常驻 GPU，而大量 LoRA 适配器存储在 CPU 或外部存储中。由于 GPU 显存有限，系统无法同时加载所有适配器，因此需要在运行过程中动态决定哪些适配器驻留在 GPU 上，哪些适配器需要被替换或重新加载。频繁的适配器加载与切换会带来显著的开销，从而影响系统的整体性能。

现有方法大多采用被动的适配器管理策略，例如基于请求触发的加载机制或简单的缓存策略（如 FIFO 或 LRU）。然而，这类方法缺乏对未来请求的预判能力，容易导致频繁的缓存失效与适配器切换。为了解决上述问题，本文提出一种基于预测模型的适配器管理方法。该方法通过分析历史请求模式与系统状态，对未来适配器的使用情况进行估计，并据此为每个适配器分配一个效用评分，从而指导适配器的驻留与淘汰决策。在显存受限的条件下，该方法能够优先保留高价值适配器，从而减少加载开销，提高系统性能。

2. 系统模型与问题定义

2.1 系统模型

本文考虑一个多租户的大语言模型推理系统，其中一个共享的基础模型被加载至 GPU 上，而多个 LoRA 适配器用于支持不同任务或用户请求。设适配器集合为 $\mathcal{A} = a_1, a_2, \dots, a_N$ ，系统在运行过程中需要处理一系列请求序列 $\mathcal{R} = r_1, r_2, \dots$ ，其中每个请求 r_i 关联唯一的适配器 $a(r_i) \in \mathcal{A}$ 。

由于 GPU 显存资源有限，系统无法同时容纳所有适配器。设 GPU 可用于适配器的显存预算为 M ，每个适配器 a 占用显存大小为 $mem(a)$ ，则任意时刻驻留在 GPU 上的适配器集合 $\mathcal{H}_t \subseteq \mathcal{A}$ 必须满足如下约束：

$$\sum_{a \in \mathcal{H}_t} mem(a) \leq M \quad (1)$$

当一个请求到达时，若其所需适配器已经存在于当前驻留集合 $\mathcal{H}_t$ 中，则可以直接执行推理；否则，系统需要将该适配器从 CPU 或外部存储加载至 GPU。该过程通常伴随着显著的时间开销，并可能触发已有适配器的淘汰操作。此外，不同适配器在显存占用与加载开销上可能存在显著差异，这进一步增加了资源管理的复杂性。因此，系统需要在运行过程中持续做出决策，以动态维护适配器驻留集合 $\mathcal{H}_t$ 。

2.2 问题定义

在上述系统模型下，本文关注如下核心问题：在给定请求序列 $\mathcal{R}$ 和显存约束 M 的条件下，如何动态管理适配器的驻留与淘汰，以优化系统整体性能。具体而言，在每个时间步 t ，系统接收到一个请求 r_t ，其对应适配器为 $a_t = a(r_t)$ 。系统需要根据当前状态（包括历史请求、当前驻留集合以及系统负载情况）决定是否加载新的适配器，并在必要时选择合适的适配器进行淘汰。该问题可以形式化为一个在线决策问题：系统在未知未来请求的情况下，基于当前及历史信息，对适配器驻留集合 $\mathcal{H}_t$ 进行动态更新，从而最小化长期运行成本。

本文将系统优化目标定义为最小化适配器管理带来的总体开销。该开销主要来源于适配器加载与切换操作。设在时间步 t 是否发生适配器加载由指示函数 $\mathbb{I}_{\text{miss}}(t)$ 表示，则总体开销可表示为：

$$\min \sum_t \mathbb{I}_{\text{miss}}(t) \cdot cost(a_t) \quad (2)$$

其中 $cost(a_t)$ 表示加载适配器 a_t 的开销，其可能与适配器大小、带宽以及系统状态相关。

该优化目标不仅影响平均延迟，还会对尾延迟（如 P95 latency）和系统吞吐量产生重要影响。此外，由于适配器大小不一致，简单地最小化缓存未命中次数并不能直接反映系统性能，因此需要在决策过程中同时考虑适配器的显存占用与加载成本。

综上所述，本文的目标是在显存约束下，通过设计合理的适配器管理策略，在适配器驻留收益与加载成本之间取得平衡，从而提升多 LoRA 推理系统的整体性能。

2.3 基线方法

为了全面评估本文所提出方法的有效性，本文选取多种具有代表性的适配器管理策略作为基线方法进行对比。这些方法涵盖了经典缓存策略中的不同设计思想，包括基于时间局部性与访问频率的策略。

2.3.1 FIFO (First-In, First-Out)

FIFO 是最简单的适配器管理策略之一。在该方法中，适配器按照进入 GPU 的时间顺序进行管理。当系统需要加载新的适配器且显存不足时，将优先淘汰最早加载到 GPU 中的适配器。该方法不利用任何访问历史信息，也不考虑适配器的使用频率或未来需求，因此属于完全被动的策略。在访问模式具有明显局部性或热点分布的场景下，FIFO 往往表现较差，但其实现简单，适合作为基础对照方法。

2.3.2 LRU (Least Recently Used)

LRU 基于时间局部性原理进行适配器管理。该方法假设近期被访问的适配器在未来更可能被再次使用。因此，当需要淘汰适配器时，系统会优先移除最长时间未被访问的适配器。具体而言，系统为每个适配器维护最近一次访问时间戳。当发生显存不足时，选择时间戳最早的适配器进行淘汰。LRU 能够有效捕捉短期访问局部性，在许多传统缓存场景中具有良好表现。然而，该方法仅依赖最近一次访问信息，无法反映访问频率或长期趋势，也无法应对突发访问模式的变化。

2.3.3 LFU (Least Frequently Used)

LFU 基于访问频率进行适配器管理。该方法通过统计每个适配器的访问次数，优先淘汰访问频率最低的适配器。在实现上，系统为每个适配器维护累计访问计数。当需要进行淘汰时，选择次数最少的适配器。LFU 能够较好地识别长期热点适配器，但其主要

局限为缺乏时间敏感性，在访问模式发生变化时，LFU 难以及时适应新的热点分布。因此，在具有动态或突发特性的工作负载中，LFU 的性能可能受到限制。

2.3.4 方法对比分析

上述三种方法均属于被动缓存策略，即仅基于历史访问信息进行决策，而不显式建模未来请求。这些方法在传统缓存场景中具有一定效果，但在多 LoRA 适配器推理环境下存在明显局限，比如未考虑适配器显存占用差异以及不具备对未来需求的预测能力。相比之下，本文方法通过引入预测机制与显存感知策略，从根本上扩展了传统缓存问题的建模方式，使其能够更好地适应多适配器推理场景中的复杂需求。

3. 方法设计

3.1 方法概述

针对多 LoRA 适配器推理中频繁加载与切换带来的性能问题，本文提出一种基于预测的适配器管理方法。该方法的核心思想是在显存受限的条件下，通过对适配器未来使用情况进行轻量级预测，动态维护一个高效的适配器驻留集合。

与传统依赖历史访问模式的被动缓存策略（如 FIFO 或 LRU）不同，本文方法显式建模适配器的未来使用概率，并基于该预测结果进行资源分配决策。具体而言，在每个时间步 t ，系统根据历史请求序列与当前系统状态，为每个适配器计算一个效用评分，并在显存约束下选择一组高效用适配器驻留在 GPU 中，从而减少不必要的加载开销。

3.2 基于 EMA 的适配器使用预测

为了刻画适配器在未来时间窗口内的使用概率，本文引入 EMA（Exponential Moving Average）对适配器访问行为进行建模。相比简单的频率统计方法，EMA 能够更好地捕捉时间序列中的动态变化，对近期访问赋予更高权重，从而更准确地反映短期趋势。

对于任意适配器 a ，其在时间步 t 的 EMA 值定义为：

$$EMA_t(a) = \lambda \cdot EMA_{t-1}(a) + (1 - \lambda) \cdot \mathbb{I}(a_t = a) \quad (3)$$

其中： $\lambda \in [0,1]$ 为衰减因子，用于控制历史信息的影响程度； $\mathbb{I}(a_t = a)$ 为指示函数，当时间步 t 的请求使用适配器 a 时取 1，否则为 0。该指标可以被视为对适配器在未来短时间内被访问概率的近似估计。通过 EMA 建模，系统能够在保持较低计算开销的同时，动态适应访问模式的变化。

3.3 适配器效用建模

在预测适配器未来使用概率的基础上，本文进一步结合系统状态信息，构建适配器的综合效用函数。对于适配器 a ，其在时间步 t 的基础效用定义为：

$$S_t(a) = \alpha \cdot EMA_t(a) + \beta \cdot Q_t(a) + \gamma \cdot A_t(a) \quad (4)$$

其中： $EMA_t(a)$ 表示适配器的预测访问概率； $Q_t(a)$ 表示当前请求队列中等待该适配器的请求数量； $A_t(a)$ 为活跃指示变量（若适配器当前正在被使用，则为 1，否则为 0）； α, β, γ 为权重参数。该效用函数融合了预测信号与即时系统需求，使得系统既能够捕捉长期趋势，又能响应短期负载变化。

3.4 显存感知的效用归一化

在多适配器推理场景中，不同适配器的显存占用存在显著差异。若仅依据访问概率进行决策，可能导致少数大规模适配器占据大量显存，从而降低整体资源利用效率。

为此，本文引入显存感知机制，对适配器效用进行归一化处理：

$$\tilde{S}_t(a) = \frac{S_t(a)}{(mem(a))^\delta} \quad (5)$$

其中： $mem(a)$ 表示适配器的显存占用； δ 为调节参数，用于控制显存因素的重要性。该设计使系统倾向于选择“单位显存收益更高”的适配器，从而在有限资源下实现更优的整体性能。

3.5 动态适配器选择策略

在每个时间步，系统根据归一化的效用 $\tilde{S}_t(a)$ 对所有适配器进行排序，并在显存约束下构建驻留集合 $\mathcal{H}_t$ ：

$$\mathcal{H}_t^* = \operatorname{argmax}_{\mathcal{H} \subseteq \mathcal{A}} \sum_{a \in \mathcal{H}} \tilde{S}_t(a) \quad \text{s.t.} \quad \sum_{a \in \mathcal{H}} mem(a) \leq M \quad (5)$$

该问题本质上是一个带容量约束的组合优化问题，类似于 0-1 背包问题。考虑到在线系统对实时性的要求，本文采用基于效用排序的贪心近似算法进行求解，以在较低计算开销下获得接近最优的解。

3.6 在线决策过程

Algorithm 1: 基于预测的适配器调度算法

Input: 当前驻留集合 $\mathcal{H}_t$; 请求 r_t 及其对应适配器 a_t ; 显存预算 M 。

Output: 更新后的驻留集合 $\mathcal{H}_{t+1}$ 。

```

1 更新适配器  $a_t$  的 EMA 值及相关统计信息;
2 计算所有适配器的效用评分  $\tilde{S}_t(a)$ ;
3 if  $a_t \in \mathcal{H}_t$  then
4   | 直接执行请求  $r_t$ ;
5 else
6   | if 显存足够加载  $a_t$  then
7   |   | 加载适配器  $a_t$ ;
8   | else
9   |   | 按  $\tilde{S}_t(a)$  从低到高排序当前驻留集合  $\mathcal{H}_t$ ;
10  |   | 依次淘汰适配器，直到满足显存约束;
11  |   | 加载适配器  $a_t$ ;
12  |   | 更新驻留集合  $\mathcal{H}_t$ ;
13 return 更新后的驻留集合  $\mathcal{H}_{t+1}$ ;

```

4. 实验设计

4.1 实验目标

本节旨在系统评估本文提出的基于预测的适配器调度方法在多 LoRA 推理场景下的有效性。我们主要关注以下几个问题：首先，与传统缓存策略（如 LRU 和 LFU）相比，所提出方法是否能够在有限显存约束下提升系统吞吐量并降低请求延迟；其次，该

方法是否能够更高效地利用显存资源，从而提升热点适配器的命中率；此外，我们进一步分析引入预测机制（如 EMA）对系统性能的具体贡献；最后，我们评估该方法在不同负载模式（包括均匀分布、长尾分布以及动态变化场景）下的稳定性与泛化能力。

4.2 实验设置

使用 NVIDIA A800（80GB）显卡，系统为 Ubuntu Linux 5.4.0，CPU 为 Intel Xeon Gold 5320 @ 2.20GHz，内存为 692GB。所有实验均在相同硬件条件下运行，以保证结果的可比性。

在模型方面，我们选用 Llama-2-13b 作为基础模型，基础模型占用显存约 28G，并构建多个 LoRA 适配器以模拟多任务推理场景。本文实验中，所有 LoRA 适配器的总显存占用约为 5GB。由于 GPU 可用于适配器的显存预算仅设置为 2GB 或 4GB，因此系统无法同时加载全部适配器，需要在运行过程中动态进行适配器驻留与淘汰决策。

4.3 工作负载

为了评估不同适配器管理策略在多 LoRA 推理场景下的表现，本文设计了三类具有代表性的工作负载，分别对应长尾访问分布、动态热点迁移以及异构适配器大小带来的显存竞争。设适配器集合为 $\mathcal{A} = \{a_1, a_2, \dots, a_M\}$ ，请求序列为 $\mathcal{R} = (r_1, r_2, \dots, r_N)$ 。每个请求 r_t 对应一个适配器 $x_t \in \mathcal{A}$ 。其中， M 表示适配器总数， N 表示请求总数。

4.3.1 长尾分布

在真实在线服务中，适配器访问通常具有明显的长尾特征，即少数适配器被频繁访问，而大量适配器只被偶尔访问。为了模拟这一现象，本文采用 Zipf 分布生成请求。

对于第 i 个适配器 a_i ，其被访问的概率定义为： $P(x_t = a_i) = \frac{i^{-s}}{\sum_{j=1}^M j^{-s}}$ ，其中 s 为 Zipf 分布参数。本文实验中设定 $s = 1.0$ 。因此，在长度为 N 的请求序列中，适配器 a_i 的期望访问次数为： $\mathbb{E}[C_i] = N \cdot P(x_t = a_i)$ 其中： $C_i = \#\{t: x_t = a_i\}$

该工作负载主要用于评估策略在长尾热点分布下的适配器保留能力。若策略能够准确识别高频适配器并优先保留它们，则可以提高缓存命中率并降低适配器加载次数。

4.3.2 动态热点迁移

在实际推理服务中，请求分布往往不是固定不变的。不同时间段内，热门适配器集合可能会发生变化，例如由于用户行为变化、任务类型变化或短时间热点事件导致访问模式转移。为模拟这种非平稳访问模式，构造动态热点迁移工作负载。

请求序列被划分为 P 个阶段，每个阶段包含约： $Q = \lfloor N/P \rfloor$ 个请求。对于第 p 个阶段，定义其热点集合为： $\mathcal{H}_p \subseteq \mathcal{A}$ 并满足： $|\mathcal{H}_p| = H$ ，其中 H 表示每个阶段的热点适配器数量。在阶段 p 中，请求以较高概率访问热点集合 $\mathcal{H}_p$ 。具体地，设热点请求比例为 θ ，则有： $P(x_t \in \mathcal{H}_p) = \theta$ 以及 $P(x_t \notin \mathcal{H}_p) = 1 - \theta$ 。本文实验中设定 $\theta = 0.8$ ，表示约 80% 的请求集中在当前阶段热点集合中。

为了体现热点迁移，相邻阶段的热点集合只有有限重叠。设重叠比例为 ρ ，则相邻热点集合满足 $|\mathcal{H}_p \cap \mathcal{H}_{p+1}| \approx \rho H$ ，当 ρ 较小时，热点集合变化更加剧烈，负载对策略的适应能力要求更高。

该工作负载主要用于评估策略面对热点变化时的自适应能力。相比 Zipf 工作负载，热点迁移更强调时间局部性和阶段切换后的恢复速度。一个有效的策略应当能够在热点变化后快速调整 GPU 中的适配器集合，从而降低阶段切换带来的缓存未命中和尾延迟。

4.3.3 异构适配器

在多 LoRA 推理系统中，不同适配器的显存占用可能存在明显差异。若策略只考虑访问频率，而忽略适配器大小，则可能会保留少数大适配器并挤占大量显存，导致更多小适配器被频繁淘汰。为评估这一问题，本文构造异构适配器工作负载。

设每个适配器 a_i 的显存占用为: $mem(a_i)$, 根据显存大小, 将适配器划分为小型/中型适配器集合 $\mathcal{S}$ 和大型适配器集合 $\mathcal{L}$, 其中 $\mathcal{S} = \{a_i: mem(a_i) \leq \eta\}$, $\mathcal{L} = \mathcal{A} \setminus \mathcal{S}$, η 为大小划分阈值。

在该工作负载中, 大部分请求访问小型或中型适配器, 少量请求周期性访问大型适配器。设访问小型/中型适配器集合的概率为 ϕ , 则有: $P(x_t \in \mathcal{S}) = \phi$ 和 $P(x_t \in \mathcal{L}) = 1 - \phi$, 本文实验中设定 $\phi = 0.75$, 表示约 75% 的请求来自小型/中型适配器集合。

该设计使系统面临显存收益与访问频率之间的冲突: 大型适配器虽然访问频率较低, 但一旦加载会占用较多 GPU 显存; 小型适配器访问更频繁, 但更容易被大适配器挤出。因此, 该工作负载主要用于评估策略是否具备显存感知能力。在该场景下, 一个有效的适配器管理策略不应只最大化访问频率, 而应综合考虑适配器收益和显存成本。

4.4 实现细节

本文实现了一个面向单基础模型、多 LoRA 适配器场景的原型推理运行时系统, 用于评估不同适配器驻留与淘汰策略在显存受限条件下的性能表现。系统整体由实验配置、运行时调度、请求执行以及结果统计等部分组成。在实验开始前, 系统首先生成多个不同规模与不同 Rank 的 LoRA 适配器, 用于模拟真实多租户场景下大小差异显著的适配器集合。随后, 系统对每个适配器进行 profiling, 记录其 GPU 显存占用 (VRAM Usage) 以及适配器加载时间 (Load Time), 并根据测量结果将适配器划分为不同大小层级。实验统一采用固定数量的适配器集合与显存预算, 包含 2GB 和 4GB 两种显存场景, 对比策略包括 FIFO、LRU、LFU 以及本文提出的策略, 同时支持本文策略的消融实验。

系统运行时由 AdapterRuntime 统一管理, 共享基础模型始终驻留于 GPU 中, 而 LoRA 适配器则由 AdapterPool 根据显存预算动态加载与淘汰。对于每一个请求, 系统首先检查目标适配器是否已经驻留在 GPU 中; 若发生缓存命中, 则直接执行推理; 否则根据当前策略选择待淘汰适配器并完成加载。FIFO、LRU 与 LFU 分别基于进入时

间、最近访问时间以及历史访问频率进行决策，而本文提出的策略则综合考虑短期趋势、长期趋势、队列压力、活跃适配器状态以及适配器显存占用等信息，对适配器计算显存感知效用分数。为了更贴近真实推理服务场景，系统采用有界 FIFO 请求队列模拟请求到达与排队过程，并记录队列等待时间、适配器加载时间、淘汰时间以及后端推理时间等阶段化延迟。实验结束后，系统会输出适配器命中率、缓存未命中次数、适配器淘汰次数、平均延迟、P95 延迟、吞吐量以及总加载时间等指标，用于分析不同适配器管理策略在统一工作负载与显存约束下的性能差异。

4.5 数据分析

我们从多个维度对实验结果进行分析。首先，我们比较不同方法在各类工作负载下的延迟与吞吐量表现，以评估整体性能差异。其次，我们分析缓存命中率与适配器替换行为，从而理解不同策略在资源管理上的差异。此外，我们通过消融实验评估预测机制的贡献，例如去除 EMA 或调整其参数，以分析各组成模块对性能的影响。最后，我们重点考察方法在动态负载场景下的表现，以验证其适应性与稳定性。

4.5.1 总体表现

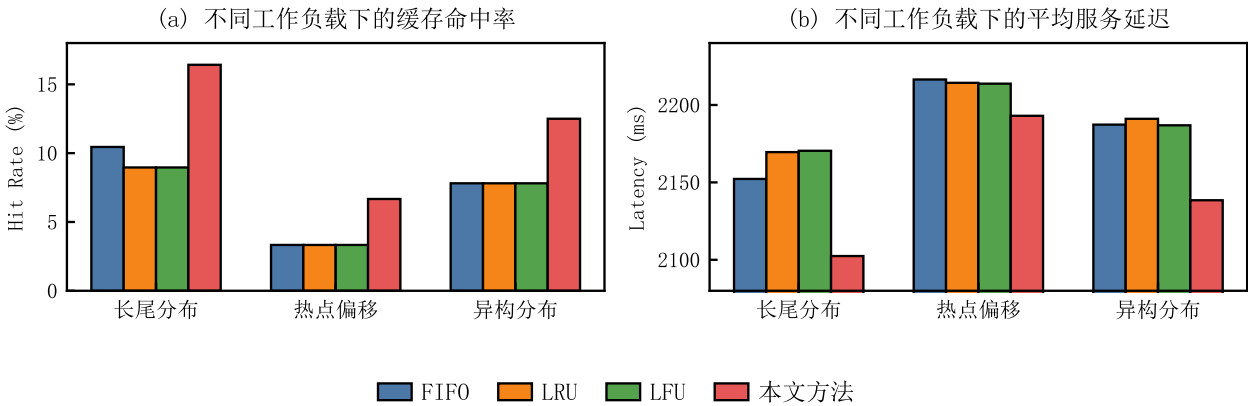


图 1 4GB 显存限制下的总体表现

图 1(a) 与图 1(b) 展示了在 4GB 显存预算下，不同适配器管理策略在多种工作负载中的缓存命中率与平均服务延迟。从缓存命中率结果可以观察到，本文提出的方法在三种典型工作负载下均取得了最优表现。

在长尾分布场景下，本文方法命中率相比 FIFO 提升明显，说明预测驱动的调度策略能够更有效地识别高频适配器并维持其驻留状态。在热点偏移场景下，由于热点集合会随时间动态变化，传统 FIFO、LRU 与 LFU 方法难以及时适应新的访问模式，因此命中率较低。而本文方法通过短期与长期访问趋势建模，能够更快响应热点迁移，从而获得更高缓存命中率。此外，在异构分布场景下，本文方法同样表现出更高命中率。这说明显存感知机制能够有效处理不同适配器之间的显存占用差异，从而提高整体资源利用效率。从平均服务延迟结果来看，本文方法在各类工作负载下均取得较低延迟。这主要得益于更高的缓存命中率与更少的适配器切换操作，从而降低了适配器加载开销。

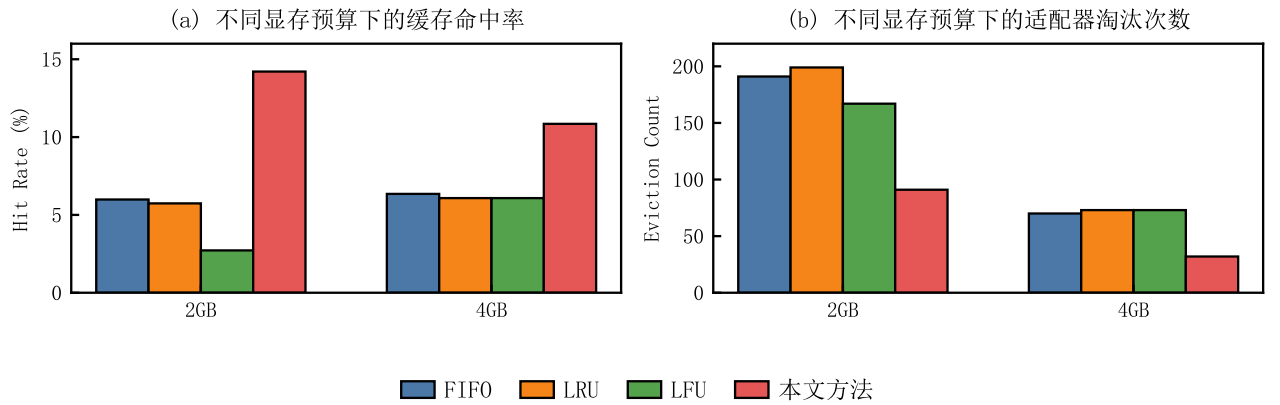


图 2 不同显存预算对比

图 2 展示了不同显存预算下，各方法在整体缓存命中率与适配器淘汰次数上的表现。可以观察到，在 2GB 与 4GB 两种显存预算下，本文方法均取得最高缓存命中率，并显著降低了适配器淘汰次数。其中，在 2GB 场景下，本文方法命中率相比 FIFO 提升超过两倍，同时适配器淘汰次数显著下降。这一结果表明，当 GPU 显存资源更加紧张时，适配器驻留决策的重要性会进一步提高，而基于预测的调度策略能够更加有效地利用有限显存资源。

4.5.2 消融实验

为了进一步分析各模块对系统性能的影响，本文进行了消融实验，结果如图 3 所示。可以观察到，移除显存归一化模块后，缓存命中率出现明显下降。这说明在多 LoRA 推理场景下，仅基于访问频率进行决策难以有效处理适配器大小差异问题。相比之下，移除短期频率项与成本感知项后，系统性能变化相对较小。这表明在当前实验设置下，长期访问模式仍然占主导作用，同时预测信号仍存在进一步优化空间。

总体而言，消融实验验证了显存感知机制对于提高系统整体资源利用效率的重要性。

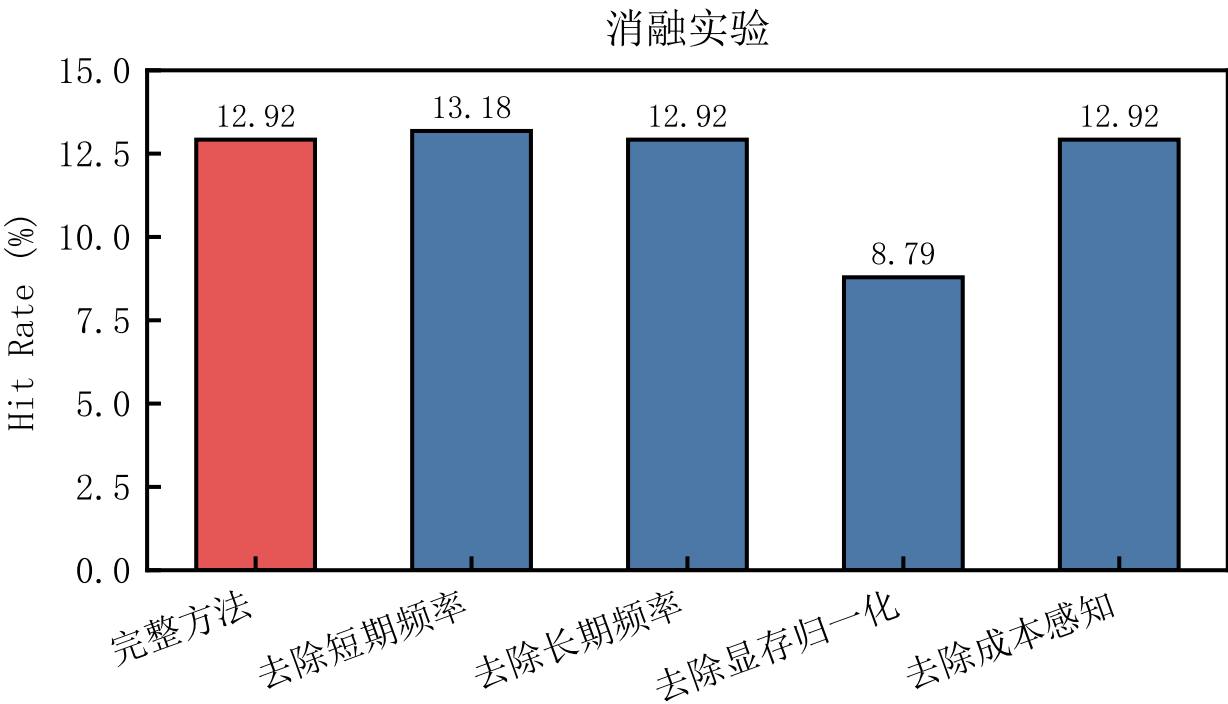


图 3 消融实验对比图

5. 相关研究

5.1 参数高效微调 (PEFT)

随着大语言模型规模的持续增长，传统全参数微调方法在训练成本与存储开销方面面临巨大挑战。为了解决这一问题，参数高效微调 (Parameter-Efficient Fine-Tuning, PEFT) 方法逐渐成为研究热点 [1-3]。早期的 Adapter 方法通过在

Transformer 层中插入少量可训练模块，在冻结主体模型参数的情况下实现任务适配 [4]。随后，Prefix-Tuning 将可训练参数进一步压缩为连续前缀向量，从而降低训练与存储成本 [5]。LoRA (Low-Rank Adaptation) 进一步提出使用低秩分解近似权重更新，仅训练少量低秩矩阵即可获得接近全参数微调的性能 [6]。由于其具有较低的显存开销与较高的部署灵活性，LoRA 已成为当前大语言模型定制化的主流方案。此后，AdaLoRA [7] 与 QLoRA [8] 等工作又分别从动态秩分配与量化训练角度进一步降低资源消耗。

然而，上述研究主要关注如何高效完成模型微调，而较少考虑多 LoRA 适配器在推理阶段的动态管理问题。随着多租户推理场景的出现，如何在有限 GPU 显存下同时服务大量 LoRA 适配器，逐渐成为新的系统挑战。

5.2 多 LoRA 推理系统

近年来，研究者开始关注多 LoRA 适配器的推理服务问题。PetS 是较早从系统角度统一管理参数高效模型的工作，其尝试通过统一表示支持多任务推理 [9]。Punica 进一步针对多租户 LoRA 推理场景进行了优化。该方法通过定制 CUDA kernel 与多租户批处理机制，实现了多个 LoRA 适配器请求的高效并行执行，从而显著提升系统吞吐量 [10]。S-LoRA 则提出 Unified Paging 与异构批处理机制，使系统能够同时服务上千个 LoRA 适配器，并有效降低适配器切换带来的性能损失 [11]。随后，dLoRA 将优化进一步扩展到运行时编排层，通过动态 merge/unmerge、请求迁移与适配器协同调度，在吞吐量与延迟方面取得进一步提升 [12]。最新的 Toppings 工作则通过 CPU-assisted prefilling 与 rank-aware scheduling 等机制，降低了适配器加载对生成阶段的影响 [13]。

总体而言，现有多 LoRA 推理系统的优化重点主要集中在 CUDA kernel、分页管理、异构批处理以及 CPU/GPU 协同等执行层面，而对“未来哪些适配器更可能再次被访问”这一问题缺乏显式建模。因此，在显存受限场景下，适配器驻留与淘汰策略仍具有较大的优化空间。

5.3 大语言模型推理与缓存管理

除了多 LoRA 推理系统之外，大语言模型推理系统本身的资源管理问题也受到广泛关注。vLLM 提出的 PagedAttention 将 KV Cache 管理转化为类似操作系统分页的问题，从而显著降低显存碎片并提高推理吞吐量 [14]。Orca [15]、Sarathi-Serve [17] 与 Splitwise [18] 等工作则分别从迭代级调度、Prefill/Decode 解耦以及请求编排等角度优化大语言模型推理性能。另一方面，在传统缓存与资源管理领域，FIFO、LRU 与 LFU 等经典缓存策略被广泛应用于对象缓存与存储系统中 [19-21]。然而，这些方法通常仅依赖历史访问行为进行决策，缺乏对未来访问模式的预测能力。

为进一步提升缓存效率，研究者提出了大量预测驱动或学习式缓存方法。例如 ARC 能够在时间局部性与访问频率之间动态平衡 [19]；TinyLFU 利用频率感知 admission 提高缓存命中率 [20]；LeCaR [22] 与 LRB [23] 则尝试利用机器学习近似最优缓存替换策略。近期的 Learned Prefix Caching 工作进一步表明，在大语言模型推理场景下，轻量预测机制能够有效降低缓存需求并提高推理效率 [27]。

受上述研究启发，本文将预测驱动思想引入多 LoRA 适配器管理问题中，通过 EMA 建模适配器访问趋势，并结合显存感知机制进行适配器驻留决策，从而提高显存利用率并降低适配器切换开销。

参考文献

- [1] 唐岸达, 林宙辰. 大模型参数高效微调方法综述: 技术、趋势与挑战[J/OL]. 自动化学报, 2026. [2026-05-07]. <https://www.aas.net.cn/>
- [2] Han Z, Gao C, Liu J, Zhang J, Zhang S Q. Parameter-Efficient Fine-Tuning for Large Models: A Comprehensive Survey[J/OL]. 2024-03-21. [2026-05-07]. <https://arxiv.org/abs/2403.14608>
- [3] Wang L, Chen S, Jiang L, 等. Parameter-efficient fine-tuning in large language models: a survey of methodologies[J]. Artificial Intelligence Review, 2025, 58(227): 1-38.
- [4] Houlsby N, Giurgiu A, Jastrzebski S, 等. Parameter-Efficient Transfer Learning for NLP[C]. Long Beach: Proceedings of the 36th International Conference on Machine Learning, 2019: 2790-2799.
- [5] Li X L, Liang P. Prefix-Tuning: Optimizing Continuous Prompts for Generation[C]. Bangkok: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics, 2021: 4582-4597.
- [6] Hu E J, Shen Y, Wallis P, 等. LoRA: Low-Rank Adaptation of Large Language Models[C]. Kigali: International Conference on Learning Representations, 2022.
- [7] Zhang Q, Chen M, Bukharin A, 等. AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning[C]. Kigali: International Conference on Learning Representations, 2023.
- [8] Dettmers T, Pagnoni A, Holtzman A, Zettlemoyer L. QLoRA: Efficient Finetuning of Quantized LLMs[C]. New Orleans: Advances in Neural Information Processing Systems, 2023: 10088-10115.
- [9] Zhou Z, Wei X, Zhang J, Sun G. PetS: A Unified Framework for Parameter-Efficient Transformers Serving[C]. Carlsbad: USENIX Annual Technical Conference, 2022: 489-504.
- [10] Chen L, Ye Z, Wu Y, 等. Punica: Multi-Tenant LoRA Serving[J]. Proceedings of Machine Learning and Systems, 2024, 6: 1-13.
- [11] Sheng Y, Cao S, Li D, 等. S-LoRA: Serving Thousands of Concurrent LoRA Adapters[J]. Proceedings of Machine Learning and Systems, 2024, 6: 296-311.

- [12] Wu B, Zhu R, Zhang Z, 等. dLoRA: Dynamically Orchestrating Requests and Adapters for LoRA LLM Serving[C]. Santa Clara: 18th USENIX Symposium on Operating Systems Design and Implementation, 2024: 911-927.
- [13] Li S, Lu H, Wu T, 等. Toppings: CPU-Assisted, Rank-Aware Adapter Serving for LLM Inference[C]. Boston: USENIX Annual Technical Conference, 2025: 612-629.
- [14] Kwon W, Li Z, Zhuang S, 等. Efficient Memory Management for Large Language Model Serving with PagedAttention[C]. Koblenz: Proceedings of the 29th Symposium on Operating Systems Principles, 2023: 611-626.
- [15] Yu G I, Jeong J S, Kim G W, Kim S, Chun B G. Orca: A Distributed Serving System for Transformer-Based Generative Models[C]. Carlsbad: 16th USENIX Symposium on Operating Systems Design and Implementation, 2022: 521-538.
- [16] Li Z, Zheng L, Zhong Y, 等. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving[C]. Boston: 17th USENIX Symposium on Operating Systems Design and Implementation, 2023: 663-679.
- [17] Agrawal A, Kedia N, Panwar A, 等. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve[C]. Santa Clara: 18th USENIX Symposium on Operating Systems Design and Implementation, 2024: 117-134.
- [18] Patel P, Choukse E, Zhang C, 等. Splitwise: Efficient Generative LLM Inference Using Phase Splitting[C]. Buenos Aires: International Symposium on Computer Architecture, 2024: 118-132.
- [19] Megiddo N, Modha D S. ARC: A Self-Tuning, Low Overhead Replacement Cache[C]. San Francisco: USENIX Conference on File and Storage Technologies, 2003: 115-130.
- [20] Einziger G, Friedman R, Manes B. TinyLFU: A Highly Efficient Cache Admission Policy[J]. ACM Transactions on Storage, 2017, 13(4): 1-31.
- [21] Beckmann N, Chen H, Cidon A. LHD: Improving Cache Hit Rate by Maximizing Hit Density[C]. Renton: 15th USENIX Symposium on Networked Systems Design and Implementation, 2018: 389-403.

[22] Vietri G, Rodriguez L V, Martinez W A, 等. Driving Cache Replacement with ML-based LeCaR[C]. Boston: 10th USENIX Workshop on Hot Topics in Storage and File Systems, 2018.

[23] Song Z, Berger D S, Li K, Lloyd W. Learning Relaxed Belady for Content Distribution Network Caching[C]. Santa Clara: 17th USENIX Symposium on Networked Systems Design and Implementation, 2020: 529-544.

[24] Yang D, Berger D S, Li K, Lloyd W. A Learned Cache Eviction Framework with Minimal Overhead[J/OL]. 2023-01-27. [2026-05-07]. <https://arxiv.org/abs/2301.11861>

[25] Zhou Z, Ning X, Hong K, 等. A Survey on Efficient Inference for Large Language Models[J/OL]. 2024-04-22. [2026-05-07]. <https://arxiv.org/abs/2404.14294>

[26] Pan J, Li G. A Survey of LLM Inference Systems[J/OL]. 2025-06-27. [2026-05-07]. <https://arxiv.org/abs/2506.22358>

[27] Yang D, Li A, Li K, Lloyd W. Learned Prefix Caching for Efficient LLM Inference[C/OL]. 2025-09-18. [2026-05-07]. <https://arxiv.org/abs/2509.15750>